In [1]:
```
# Phase 01 - Ingestion, Profiling & IT Filtering (FIXED - Windows Compatible)
# ==============================================================================
# Goals:
# - Load raw datasets with explicit dtypes and stable IDs
# - Basic profiling (shape, nulls, uniques, length stats)
# - Filter job posts to IT (rule-based + semantic)
# - Near-duplicate pruning (5-gram Jaccard > 0.95)
# - Save clean artifacts via standard Report helper
#
# FIXES APPLIED:
# 1. [OK] Resume ID duplicate detection and resolution
# 2. [OK] Streamlined workflow (removed redundant cells)
# 3. [OK] Single near-duplicate function (no redefinition)
# 4. [OK] Column normalization done upfront
# 5. [OK] Added ontology duplicate detection
# 6. [OK] Better validation and error checking
# 7. [OK] NO EMOJI - Windows console compatible
# ==============================================================================

# %% [markdown]
# # Phase 01 - Ingestion, Profiling & IT Filtering
#
# **Inputs:**
# - `data/raw/resume_dataset_1200.csv`
# - `data/raw/all_job_post.csv`
# - `data/raw/IT_Job_Roles_Skills.csv`
#
# **Outputs:**
# - `outputs/phase01/tables/resumes_profile.csv`
# - `outputs/phase01/tables/jobs_profile.csv`
# - `outputs/phase01/tables/ontology_profile.csv`
# - `outputs/phase01/tables/jobs_it.csv` (final IT jobs after deduplication)
# - `outputs/phase01/tables/resumes_clean.csv`
# - `outputs/phase01/logs/run.log`
```

In [2]:
```
# %% Cell 1: Setup and Imports
import sys, os
from pathlib import Path
```

```python
# Make notebooks see your src/ folder
proj_root = Path.cwd()
src_path = proj_root / "src"
assert src_path.exists(), f"missing: {src_path}"
sys.path.append(str(src_path))

print("Python:", sys.executable)
print("Added to sys.path:", src_path)

# Standard phase header
from common.seed_logging import set_global_seed, get_logger
from common.reporting import Report
from common.paths import PathResolver

set_global_seed(42)
log = get_logger("phase01")
R = Report(phase=1)
P = PathResolver()
R.stamp("Phase 01 start")

# Other imports
import pandas as pd
import numpy as np
import re
from typing import Iterable, Set
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

print("[OK] Setup complete")
```

```
Python: c:\Users\naikt\OneDrive\Desktop\tanmay_dissertation\.venv\Scripts\python.exe
Added to sys.path: c:\Users\naikt\OneDrive\Desktop\tanmay_dissertation\src
[OK] Setup complete
```

In [3]:

```python
# %% Cell 2: Helper Functions
def profile_df(df: pd.DataFrame, name: str) -> pd.DataFrame:
    """Return a tidy profile for quick inspection."""
    nulls = df.isna().sum().rename("null_count")
    uniques = df.nunique(dropna=True).rename("nunique")
    out = pd.concat([nulls, uniques], axis=1).reset_index().rename(columns={"index": "column"})
    out.insert(1, "dtype", [str(df[c].dtype) for c in out["column"]])
```

```python
    out.insert(0, "dataset", name)

    # Length stats for text columns
    lens = {}
    for c in df.columns:
        if df[c].dtype == "object":
            s = df[c].astype(str).fillna("")
            lens[c] = s.str.len().describe()[["mean","50%","max"]].rename({"50%": "median"})
    if lens:
        lens_df = pd.DataFrame(lens).T.reset_index().rename(columns={"index":"column"})
        lens_df.columns = ["column","len_mean","len_median","len_max"]
        out = out.merge(lens_df, on="column", how="left")
    return out

def add_stable_id(df: pd.DataFrame, cols: Iterable[str], new_col: str) -> pd.DataFrame:
    """Create reproducible hash ID from selected columns."""
    import hashlib
    temp = df[list(cols)].astype(str).fillna("")
    key = temp.apply(lambda r: "||".join(r.values.tolist()), axis=1).str.encode("utf-8")
    df[new_col] = [hashlib.md5(x).hexdigest() for x in key]
    return df

def _char_ngrams(text: str, n: int = 5) -> Set[str]:
    """Extract character n-grams from text."""
    text = (text or "").lower().strip()
    if len(text) < n:
        return {text} if text else set()
    return {text[i:i+n] for i in range(len(text)-n+1)}

def jaccard_5gram(a: str, b: str, n: int = 5) -> float:
    """Calculate Jaccard similarity using character 5-grams."""
    A, B = _char_ngrams(a, n), _char_ngrams(b, n)
    if not A and not B:
        return 1.0
    if not A or not B:
        return 0.0
    return len(A & B) / len(A | B)

def near_duplicate_mask(texts: pd.Series, threshold: float = 0.95) -> pd.Series:
    """
    Return boolean mask for rows to KEEP, removing near-duplicates.
```

```
    Uses greedy sequential comparison with already-kept exemplars.
    """
    s = texts.fillna("").astype(str)
    orig_index = s.index
    s_pos = s.reset_index(drop=True)  # positional indexing
    vals = s_pos.tolist()

    keep = np.ones(len(s_pos), dtype=bool)
    kept_pos = []

    for i, t in enumerate(vals):
        if not keep[i]:
            continue
        # Compare with already-kept exemplars
        is_dup = any(jaccard_5gram(t, vals[j]) >= threshold for j in kept_pos)
        if is_dup:
            keep[i] = False
        else:
            kept_pos.append(i)

    return pd.Series(keep, index=orig_index, dtype=bool)

def norm_cols(df: pd.DataFrame) -> pd.DataFrame:
    """Normalize column names to lowercase and strip whitespace."""
    df = df.copy()
    df.columns = [str(c).strip().lower() for c in df.columns]
    return df

def col_or_blank(df: pd.DataFrame, candidates: list) -> pd.Series:
    """Return first matching column from candidates, or blank Series."""
    for c in candidates:
        if c in df.columns:
            return df[c].astype(str).fillna("")
    return pd.Series([""] * len(df), index=df.index, dtype="object")

print("[OK] Helper functions defined")
```

```
[OK] Helper functions defined
```

In [4]:

```
# %% Cell 3: Robust CSV Loader
def detect_bom_encoding(path: Path):
```

```python
    """Detect BOM encoding markers."""
    with open(path, "rb") as f:
        raw = f.read(4)
    if raw.startswith(b"\xff\xfe"):
        return "utf-16le"
    if raw.startswith(b"\xfe\xff"):
        return "utf-16be"
    if raw.startswith(b"\xef\xbb\xbf"):
        return "utf-8-sig"
    return None

def read_csv_flex(path: Path, dtype_hint: dict = None):
    """Robust CSV reader that tries multiple encodings and separators."""
    enc_bom = detect_bom_encoding(path)
    trials = []
    if enc_bom:
        trials.append((enc_bom, None))

    encs = ["utf-16", "utf-8-sig", "cp1252", "latin1", "utf-8"]
    seps = [None, ",", "\t", ";", "|"]
    for e in encs:
        for s in seps:
            if (e, s) not in trials:
                trials.append((e, s))

    last_err = None
    for enc, sep in trials:
        try:
            # Read small sample first
            head = pd.read_csv(path, nrows=200, encoding=enc, sep=sep,
                               engine="python", on_bad_lines="skip")
            cols = list(head.columns)
            use_dtype = {k:v for k,v in (dtype_hint or {}).items() if k in cols}

            # Read full file
            df = pd.read_csv(path, encoding=enc, sep=sep, dtype=use_dtype,
                             engine="python", on_bad_lines="skip")
            print(f"[OK] [read OK] {path.name} | encoding={enc} | sep={repr(sep)} | shape={df.shape}")
            return df
        except Exception as e:
            last_err = e
```

```
            continue

    raise RuntimeError(f"[ERROR] Could not read {path}. Last error: {last_err}")

print("[OK] Robust CSV loader defined")
```

```
[OK] Robust CSV loader defined
```

In [5]:

```
# %% Cell 4: Load Datasets
print("\n" + "="*70)
print("LOADING DATASETS")
print("="*70)

path_resumes = P.dataset_path("resumes")
path_jobs = P.dataset_path("jobs")
path_ontology = P.dataset_path("ontology")

print(f"\nReading:\n  {path_resumes}\n  {path_jobs}\n  {path_ontology}\n")

# Load with flexible reader
resumes = read_csv_flex(path_resumes)
jobs = read_csv_flex(path_jobs)
ontology = read_csv_flex(path_ontology)

print(f"\n[OK] Loaded: resumes={resumes.shape}, jobs={jobs.shape}, ontology={ontology.shape}")
R.stamp(f"Loaded resumes={resumes.shape}, jobs={jobs.shape}, ontology={ontology.shape}")
```

```
======================================================================
LOADING DATASETS
======================================================================

Reading:
  C:\Users\naikt\OneDrive\Desktop\tanmay_dissertation\data\raw\resume_dataset_1200.csv
  C:\Users\naikt\OneDrive\Desktop\tanmay_dissertation\data\raw\all_job_post.csv
  C:\Users\naikt\OneDrive\Desktop\tanmay_dissertation\data\raw\IT_Job_Roles_Skills.csv

[OK] [read OK] resume_dataset_1200.csv | encoding=utf-8-sig | sep=None | shape=(1200, 14)
[OK] [read OK] all_job_post.csv | encoding=utf-8-sig | sep=None | shape=(1167, 5)
[OK] [read OK] IT_Job_Roles_Skills.csv | encoding=cp1252 | sep=None | shape=(493, 4)

[OK] Loaded: resumes=(1200, 14), jobs=(1167, 5), ontology=(493, 4)
```

In [6]:
```
# %% Cell 5: Normalize Column Names FIRST
print("\n" + "="*70)
print("NORMALIZING COLUMN NAMES")
print("="*70)

resumes = norm_cols(resumes)
jobs = norm_cols(jobs)
ontology = norm_cols(ontology)

print("\n[OK] Resumes columns:", list(resumes.columns)[:10])
print("[OK] Jobs columns:", list(jobs.columns)[:10])
print("[OK] Ontology columns:", list(ontology.columns)[:10])

# Define column candidates (after normalization)
TITLE_CANDS = ["job_title", "title", "role", "position", "jobtitle"]
DESC_CANDS = ["job_description", "description", "desc", "jd", "details", "responsibilities"]
CAT_CANDS = ["category", "job_category", "department", "function"]
```

```
======================================================================
NORMALIZING COLUMN NAMES
======================================================================

[OK] Resumes columns: ['name', 'age', 'gender', 'education_level', 'field_of_study', 'degrees', 'institute_name', 'graduation_y
ear', 'experience_years', 'current_job_title']
[OK] Jobs columns: ['job_id', 'category', 'job_title', 'job_description', 'job_skill_set']
[OK] Ontology columns: ['job title', 'job description', 'skills', 'certifications']

NORMALIZING COLUMN NAMES
======================================================================

[OK] Resumes columns: ['name', 'age', 'gender', 'education_level', 'field_of_study', 'degrees', 'institute_name', 'graduation_y
ear', 'experience_years', 'current_job_title']
[OK] Jobs columns: ['job_id', 'category', 'job_title', 'job_description', 'job_skill_set']
[OK] Ontology columns: ['job title', 'job description', 'skills', 'certifications']
```

In [7]:
```
# %% Cell 6: Create Stable IDs & Remove Exact Duplicates
print("\n" + "="*70)
print("CREATING STABLE IDs & REMOVING EXACT DUPLICATES")
print("="*70)
```

```python
# RESUMES: Create or verify resume_id
if "resume_id" not in resumes.columns:
    src_cols = [c for c in ["name", "skills", "education_level", "field_of_study"]
                if c in resumes.columns][:3]
    resumes = add_stable_id(resumes, src_cols, "resume_id")
    print(f"[OK] Created resume_id from columns: {src_cols}")
else:
    print("[OK] resume_id column already exists")

# Check for duplicate IDs BEFORE deduplication
id_counts = resumes['resume_id'].value_counts()
duplicates = id_counts[id_counts > 1]
if len(duplicates) > 0:
    print(f"\n[WARN] WARNING: Found {len(duplicates)} duplicate resume IDs!")
    print(f"       Duplicate IDs: {list(duplicates.index[:5])}")
    print(f"       These will be handled by deduplication...")

resumes_before = len(resumes)
resumes = resumes.drop_duplicates(subset=['resume_id'], keep='first')
resumes_removed = resumes_before - len(resumes)
print(f"[OK] Resumes: removed {resumes_removed} exact duplicates")
R.stamp(f"Resumes dropped {resumes_removed} exact duplicates")

# JOBS: Create or verify job_id
if "job_id" not in jobs.columns:
    src_cols = [c for c in ["job_title", "job_description", "category"]
                if c in jobs.columns][:3]
    jobs = add_stable_id(jobs, src_cols, "job_id")
    print(f"[OK] Created job_id from columns: {src_cols}")
else:
    print("[OK] job_id column already exists")

jobs_before = len(jobs)
jobs = jobs.drop_duplicates(subset=['job_id'], keep='first')
jobs_removed = jobs_before - len(jobs)
print(f"[OK] Jobs: removed {jobs_removed} exact duplicates")
R.stamp(f"Jobs dropped {jobs_removed} exact duplicates")

# ONTOLOGY: Create or verify role_id
if "role_id" not in ontology.columns:
    src_cols = [c for c in ["job title", "skills", "certifications"]
```

```
                if c in ontology.columns][:3]
    ontology = add_stable_id(ontology, src_cols, "role_id")
    print(f"[OK] Created role_id from columns: {src_cols}")
else:
    print("[OK] role_id column already exists")

# CHECK FOR DUPLICATE ROLES IN ONTOLOGY (addresses cosine=1.0 issue)
onto_before = len(ontology)
ontology = ontology.drop_duplicates(subset=['role_id'], keep='first')
onto_removed = onto_before - len(ontology)
if onto_removed > 0:
    print(f"[WARN] Ontology: removed {onto_removed} duplicate roles!")
    R.stamp(f"[WARN] Ontology had {onto_removed} duplicates removed")
else:
    print(f"[OK] Ontology: no duplicates found")

print(f"\nFinal shapes: resumes={resumes.shape}, jobs={jobs.shape}, ontology={ontology.shape}")
```

```
======================================================================
CREATING STABLE IDs & REMOVING EXACT DUPLICATES
======================================================================
[OK] Created resume_id from columns: ['name', 'skills', 'education_level']
[OK] Resumes: removed 0 exact duplicates
[OK] job_id column already exists
[OK] Jobs: removed 0 exact duplicates
[OK] Created role_id from columns: ['job title', 'skills', 'certifications']
[WARN] Ontology: removed 7 duplicate roles!

Final shapes: resumes=(1200, 15), jobs=(1167, 5), ontology=(486, 5)
```

In [8]:

```
# %% Cell 7: Basic Profiling
print("\n" + "="*70)
print("PROFILING DATASETS")
print("="*70)

prof_res = profile_df(resumes, "resumes")
prof_jobs = profile_df(jobs, "jobs")
prof_ont = profile_df(ontology, "ontology")

R.save_df(prof_res, "resumes_profile.csv", index=False)
R.save_df(prof_jobs, "jobs_profile.csv", index=False)
```

```
R.save_df(prof_ont, "ontology_profile.csv", index=False)

print("\n[OK] Saved profiling tables")
print("\nSample - Resumes Profile:")
print(prof_res.head())
```

```
======================================================================
PROFILING DATASETS
======================================================================

[OK] Saved profiling tables

Sample - Resumes Profile:
   dataset           column   dtype  null_count  nunique   len_mean  \
0  resumes             name  object           0      967  12.530000
1  resumes              age   int64           0       25        NaN
2  resumes           gender  object           0        3   6.873333
3  resumes  education_level  object           0        6   8.326667
4  resumes   field_of_study  object           0       38  15.921667

   len_median  len_max
0        12.0     21.0
1         NaN      NaN
2         6.0     10.0
3         8.0     11.0
4        16.0     23.0
```

In [9]:

```
# %% Cell 8: IT Filtering (Rule-Based + Semantic)
print("\n" + "="*70)
print("IT FILTERING")
print("="*70)

# Rule-based keywords
IT_KEYWORDS_TITLE = [
    "data", "software", "devops", "cloud", "ml", "ai", "security",
    "frontend", "front-end", "backend", "back-end", "fullstack", "full-stack",
    "engineer", "developer", "architect", "analyst", "scientist", "platform",
    "sre", "site reliability", "qa", "test", "android", "ios", "mobile",
    "etl", "warehouse", "bi ", "business intelligence", "database", "sql", "big data"
]
```

```python
IT_KEYWORDS_DESC = [
    "python", "java", "c++", ".net", "node", "react", "angular", "kubernetes", "docker",
    "terraform", "aws", "azure", "gcp", "spark", "hadoop", "airflow", "ci/cd", "pipeline",
    "postgres", "mysql", "nosql", "mongodb", "pytorch", "tensorflow", "scikit-learn"
]

# Extract text columns
title_s = col_or_blank(jobs, TITLE_CANDS)
desc_s = col_or_blank(jobs, DESC_CANDS)
cat_s = col_or_blank(jobs, CAT_CANDS)

# Rule-based filter
def is_it_row(t, d, c):
    t, d, c = t.lower(), d.lower(), c.lower()
    hit_cat = any(k in c for k in ["it", "software", "technology", "tech", "information-technology"])
    hit_title = any(k in t for k in IT_KEYWORDS_TITLE)
    hit_desc = any(k in d for k in IT_KEYWORDS_DESC)
    return hit_cat or hit_title or hit_desc

rule_mask = [is_it_row(t, d, c) for t, d, c in zip(title_s, desc_s, cat_s)]
print(f"[OK] Rule-based filter: {sum(rule_mask)} / {len(jobs)} jobs passed")

# Semantic filter (TF-IDF cosine similarity)
corpus = (title_s.fillna("") + " " + desc_s.fillna("")).astype(str)
anchors = [
    "software engineer developer programming coding cloud devops sre site reliability",
    "data engineer data scientist machine learning ml ai analytics python sql etl pipeline",
    "cybersecurity security engineer penetration tester identity governance",
    "frontend backend fullstack web react angular node java microservices"
]
anchor_text = anchors + corpus.tolist()

tfidf = TfidfVectorizer(min_df=3, ngram_range=(1, 2))
X = tfidf.fit_transform(anchor_text)
A = X[:len(anchors)]
J = X[len(anchors):]

scores = cosine_similarity(J, A).max(axis=1)
gate = 0.30
semantic_mask = scores >= gate
print(f"[OK] Semantic filter: {semantic_mask.sum()} / {len(jobs)} jobs passed (threshold={gate})")
```

```
# Combine filters (OR logic)
combined_mask = np.array(rule_mask) | semantic_mask
jobs_it = jobs.loc[combined_mask].copy()
print(f"[OK] Combined filter: {len(jobs_it)} / {len(jobs)} jobs retained")
R.stamp(f"IT filtering retained {len(jobs_it)} / {len(jobs)} jobs")
```

```
======================================================================
IT FILTERING
======================================================================
[OK] Rule-based filter: 609 / 1167 jobs passed
[OK] Semantic filter: 0 / 1167 jobs passed (threshold=0.3)
[OK] Combined filter: 609 / 1167 jobs retained
```

In [10]:

```
# %% Cell 9: Near-Duplicate Removal
print("\n" + "="*70)
print("NEAR-DUPLICATE REMOVAL")
print("="*70)

# Create combined text for deduplication
title_s = col_or_blank(jobs_it, TITLE_CANDS)
desc_s = col_or_blank(jobs_it, DESC_CANDS)
text_for_dupe = (title_s.fillna("") + " || " + desc_s.fillna("")).astype(str)

if text_for_dupe.str.strip().eq("").all():
    print("[WARN] No usable text found; skipping near-duplicate pruning")
    jobs_it_dedup = jobs_it.copy()
else:
    keep_mask = near_duplicate_mask(text_for_dupe, threshold=0.95)
    jobs_it_dedup = jobs_it.loc[keep_mask].copy()
    removed = len(jobs_it) - len(jobs_it_dedup)
    print(f"[OK] Removed {removed} near-duplicates (Jaccard threshold=0.95)")
    R.stamp(f"Near-duplicate pruning removed {removed} rows")

print(f"[OK] Final IT jobs: {len(jobs_it_dedup)}")
```

```
======================================================================
NEAR-DUPLICATE REMOVAL
======================================================================
[OK] Removed 9 near-duplicates (Jaccard threshold=0.95)
[OK] Final IT jobs: 600
```

In [11]:
```
# %% Cell 10: Save Clean Outputs
print("\n" + "="*70)
print("SAVING CLEAN OUTPUTS")
print("="*70)

R.save_df(jobs_it_dedup, "jobs_it.csv", index=False)
R.save_df(resumes, "resumes_clean.csv", index=False)

print(f"[OK] Saved jobs_it.csv ({len(jobs_it_dedup)} rows)")
print(f"[OK] Saved resumes_clean.csv ({len(resumes)} rows)")
R.stamp("Phase 01 complete")
```

```
======================================================================
SAVING CLEAN OUTPUTS
======================================================================
[OK] Saved jobs_it.csv (600 rows)
[OK] Saved resumes_clean.csv (1200 rows)
```

In [12]:
```
# %% Cell 11: Validation Report
print("\n" + "="*70)
print("VALIDATION REPORT")
print("="*70)

tab = P.phase_dir(1)["tab"]
files = sorted([f.name for f in Path(tab).glob("*.csv")])
print(f"\n[OK] Files saved in {tab}:")
for f in files:
    df_temp = pd.read_csv(tab / f)
    print(f"   - {f:<30} ({len(df_temp):>6} rows)")

# Verify key outputs
jobs_final = pd.read_csv(tab / "jobs_it.csv")
resumes_final = pd.read_csv(tab / "resumes_clean.csv")

print(f"\n[OK] FINAL COUNTS:")
print(f"   - Original jobs: {1167}")
print(f"   - IT jobs (after filters & dedup): {len(jobs_final)}")
print(f"   - Resumes (cleaned): {len(resumes_final)}")
print(f"   - Unique resume IDs: {resumes_final['resume_id'].nunique()}")
print(f"   - Ontology roles: {len(ontology)}")
```

```python
# Check for any remaining issues
if resumes_final['resume_id'].nunique() < len(resumes_final):
    print(f"\n[WARN] WARNING: Still have duplicate resume IDs!")
else:
    print(f"\n[OK] All resume IDs are unique")

print("\n" + "="*70)
print("[SUCCESS] PHASE 01 COMPLETE")
print("="*70)
```

```
======================================================================
VALIDATION REPORT
======================================================================

[OK] Files saved in C:\Users\naikt\OneDrive\Desktop\tanmay_dissertation\outputs\phase01\tables:
   - jobs_it.csv                    (   600 rows)
   - jobs_it_dedup.csv              (   600 rows)
   - jobs_it_filtered.csv           (   609 rows)
   - jobs_profile.csv               (     5 rows)
   - ontology_profile.csv           (     5 rows)
   - resumes_clean.csv              (  1200 rows)
   - resumes_profile.csv            (    15 rows)
   - tiny_table.csv                 (     3 rows)

[OK] FINAL COUNTS:
   - Original jobs: 1167
   - IT jobs (after filters & dedup): 600
   - Resumes (cleaned): 1200
   - Unique resume IDs: 1200
   - Ontology roles: 486

[OK] All resume IDs are unique

======================================================================
[SUCCESS] PHASE 01 COMPLETE
======================================================================
```